

Shmoo

Last updated by | Zuriel, Zamir | Jul 14, 2024 at 1:47 AM CST

prime Test-Method Specification REP

Revision 1.1.1

Apr 2022

Contents

- [REP for Shmoo](#)
- [Methodology](#)
 - [Verify\(\)](#)
 - [Execute\(\)](#)
- [Test Instance Parameters](#)
 - [AxisFormats](#)
 - [ExecutionMode](#)
- [Console output](#)
- [Datalog output](#)
 - [ShmooHub Example:](#)
 - [ECADS Example:](#)
 - [ARIES Example:](#)
- [Custom User Code Hooks](#)
 - [Extensions used in verify:](#)
 - [Extensions used in Execute](#)
- [Optional Features](#)
 - [User Defined Shmoo Axis](#)
 - [Voltage Configuration Axis](#)
 - [Regional Kill](#)
 - [Tname Scale customization](#)
 - [Axis name customization](#)
- [TPL Samples](#)
- [Exit Ports](#)
- [Additional Dependencies](#)
- [Version tracking](#)
- [Acronyms](#)

REP for Shmoo

This **REP** is intended to describe the Shmoo Prime TestMethod.

In this document, you will find the below sections:

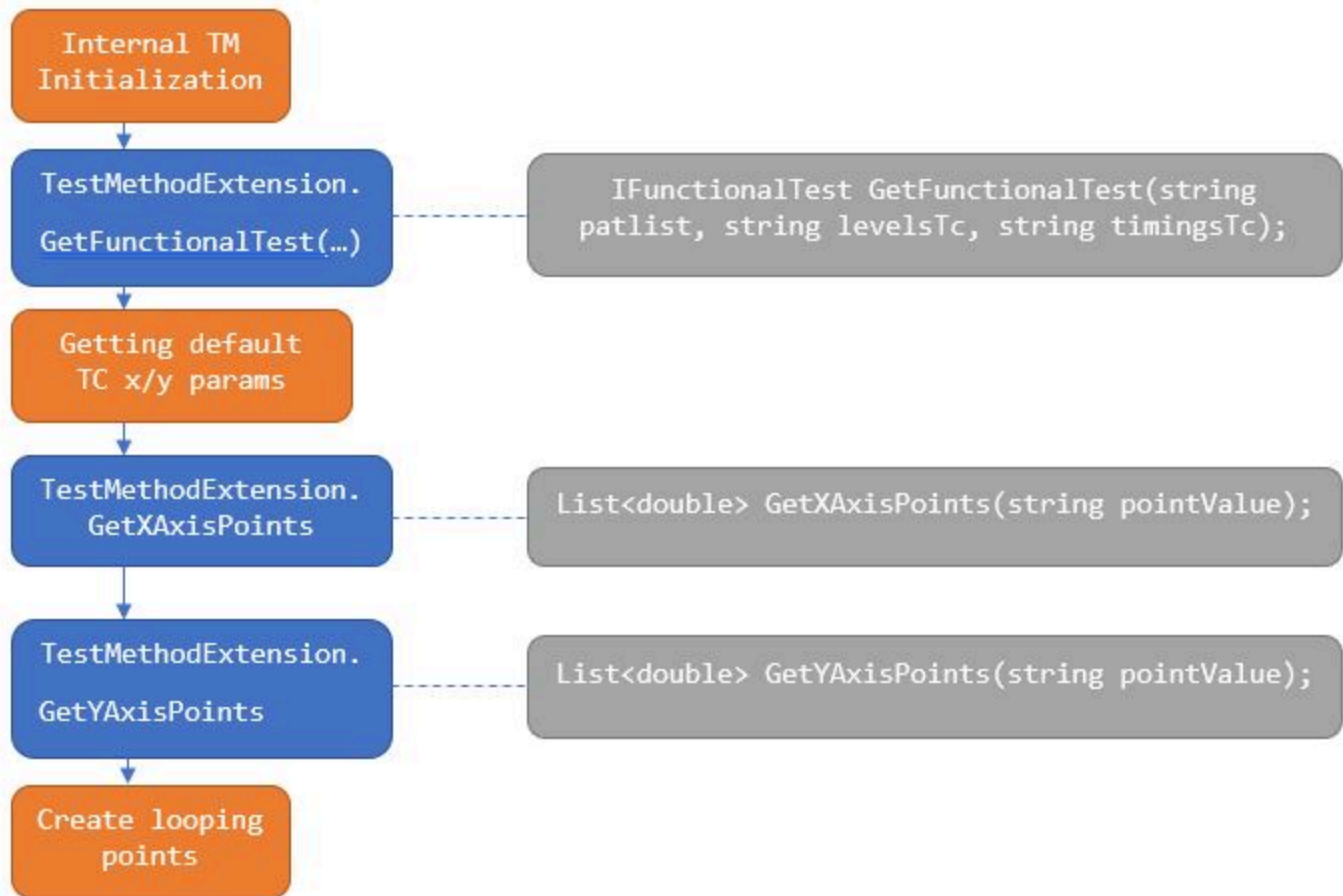
- **Methodology** – A detailed description of this TestMethod intention and purpose
- **Parameters** – A table describes each instance parameter (Name, Type, Default, Required?)
- **Datalog output** – A detailed description of what is datalogged by his TestMethod
- **Custom User Code hooks** – A list of functions available to the user code to override
- **Optional Features** – Optional Features of this test method.
- **TPL Samples** – Examples of how to use this TestMethod in a TPL file
- **Exit Ports** - A table describes each exit port
- **Additional Dependencies** – More to consider for this TestMethod to operate
- **Version tracking** – With author names, so you always have a name to address
- **Acronyms** - Definition of acronyms used in this document

Methodology

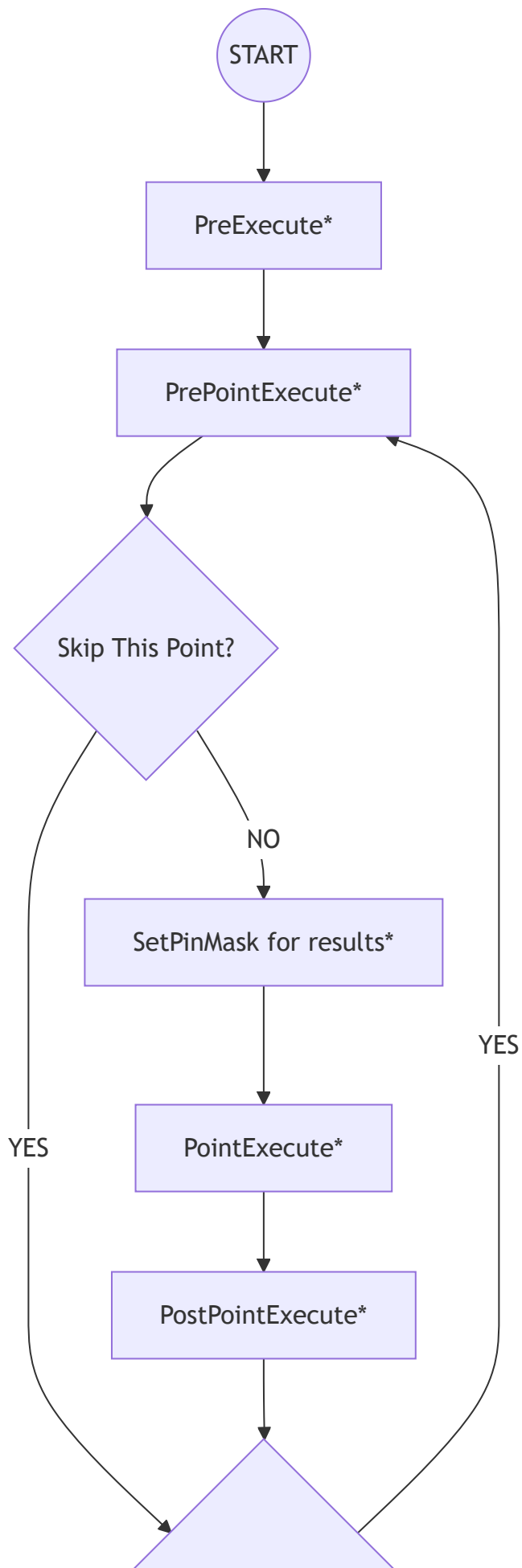
Shmoo test method provides capability to perform two dimensional iterations over range of x/y points while executing the provided plist at each point. User has the ability before and after the plist execution, to perform specific logic on the specific x/y points and the functional test object.

Flow of Execution

Verify()



Execute()

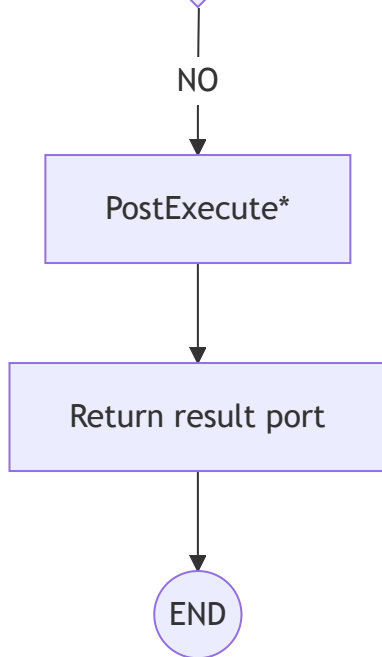


- Starred items are extendable via user code extensions.

More Points To Execute?

Test Instance Parameters

The table below lists and describes the test instance parameters supported by the Shmoo test method



Parameter Name	Required?	Type	Values	C
Patlist	Yes	Plist	Plist name to be executed at each Shmoo point	
LevelsTc	Yes	LevelsCondition	Levels test condition required for plist execution	
TimingTc	Yes	TimingCondition	Timing test condition required for plist execution	
MaskPins	No	String	Comma separated list of pins for which the fail data capture will be skipped	[S
XAxisParam	No*	String	This is the X parameter to apply value on, before executing the plist at each iteration.	* t r t l r i c f C l a f
XAxisRange	Yes if XAxisParam is provided	String	This string is passed to GetXAxisPoints(...) method to generate the x-points. By default, if user doesn't extend GetXAxisPoints(...) method, the expected format is: "Start: Resolution: NumberOfPoints" for TestCondition or PatConfig type Axes, and a comma separated list of points for PatConfigSetpoint (ex "400MHz,500MHz,600MHz,800MHz").	M s i v e
XAxisParamType	Yes if XAxisParam is provided	String (choice)	None (default), UserDefined, LevelsTestCondition, TimingTestCondition, PatConfig, PatConfigSetpoint, or VoltageConfig	

Parameter Name	Required?	Type	Values	C
XAxisDatalogPrefix	No	String (choice)	Base (default), Nano, Micro, or Milli	T C S t i r r e
XAxisDatalogName	No	String	This string is used during datalog prints to label the X axis. Useful for when XAxisParam is too long for tname prints	[S X
YAxisParam	No	String	This is the Y parameter to apply value on, before executing the plist at each iteration.	
YAxisRange	Yes if YAxisParam is provided	String	This string is passed to GetYAxisPoints(...) method to generate the y-points.	E C C r e i F N
YAxisParamType	Yes if YAxisParam is provided	String (choice)	None (default), UserDefined, LevelsTestCondition, TimingTestCondition, PatConfig, PatConfigSetpoint, or VoltageConfig	
YAxisDatalogPrefix	No	String (choice)	Base (default), Nano, Micro, or Milli	T C S t i r r e
YAxisDatalogName	No	String	This string is used during datalog prints to label the Y axis. Useful for	[S

Parameter Name	Required?	Type	Values	C
			when YAxisParam is too long for tname prints	Y
PowerDownBetweenPoints	No	String (choice)	ENABLED or DISABLED. Determines whether or not a powerdown is applied after point execution.	
PlotMode	No	String (choice)	Normal (default) or Custom. Normal mode will use the format defined in the PrintFormat parameter. Custom mode will give the user the control over symbols, legend keys and info to be printed through an extension method. The format used for Custom mode is ShmooHub but with some strict formatting on the legend keys.	
PrintFormat	No	String (choice)	ShmooHub (default), ARIES, or ECADS. ShmooHub mode will use ShmooHub format. ARIES will follow ARIES print format. ECADS will follow ECADS print format.	
RegionalKillLimits	No	Comma separated list of strings "XMax, XMin, YMax, YMin"	Inclusive limits used for Regional Kill. If using 1D shmoo, only x values are required.	
ExecutionMode	No	String (choice)	AllPin (default), PerPin.	
ShmooPins	No	Comma separated list of strings	It must be defined if ExecutionMode is PerPin.	

AxisFormats

Shmoo AxisType	Required Param Format	Comments
None	N/A	
UserDefined	N/A	A UserDefined Axis will not execute anything unless overwritten by usercode
TimingTestCondition	Timing Test conditions to apply to	
LevelsTestCondition	Levels Test conditions to apply to	
PatConfig	Name of Configurations as defined in user's *patmod.json file	
PatConfigSetpoint	name of Module:Group pairs as defined in user's *PatConfigSetpoints.json file	Notes on PatConfig related setup files

ExecutionMode

Shmoo AxisType	Required Param Format	Comments
AllPin	Execute shmoo with all pins.	
PerPin	Execute shmoo with this pins defined in Test instance parameter ShmooPins.	

Console output

Example1 (ExecutionMode = "AllPin"):

Y\X	0.4	0.5	0.6	0.7
0.2	*	*	*	*
0.4	-	*	*	*
0.6	*	*	-	*
0.8	*	*	*	*

Example1 (ExecutionMode = "PerPin"; ShmooPins = "xxHPCC_DPIN_Dig_slcA_AA0,xxHPCC_DPIN_Dig_slcA_AA1"):

Shmoo plot for pin=xxHPCC_DPIN_Dig_slcA_AA0					
Y\X	0	1E-06	2E-06	3E-06	
0	B	B	B	*	
1E-06	C	C	*	*	
2E-06	E	*	*	*	
3E-06	*	*	*	*	

Shmoo plot for pin=xxHPCC_DPIN_Dig_slcA_AA1					
Y\X	0	1E-06	2E-06	3E-06	
0	A	A	A	*	
1E-06	D	D	*	*	
2E-06	*	*	*	*	
3E-06	*	*	*	*	

Datalog output

ShmooHub Example:

Example1 (ExecutionMode = "AllPin"):

```
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensional_P1_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensional_P1_ShmooParams
2_strgval_drive_var1^0^3E-06^1E-06^drive_var2^0^3E-06^1E-06_ShmooHub
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensional_P1_ShmooResults
2_strgval_**** *
2_tname_Shmoo::PrimeShmooTestMethodOneDimensional_P1_SSTP
2_strgval_X_10e-9S
2_tname_Shmoo::PrimeShmooTestMethodOneDimensional_P1_ShmooParams
2_strgval_drive_var1^0^3E-06^1E-06_ShmooHub
2_tname_Shmoo::PrimeShmooTestMethodOneDimensional_P1_ShmooResults
2_strgval_****
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0_SSTP
2_strgval_X_10e-9S
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0_ShmooParams
2_strgval_drive_var1^0^3^1_ShmooHub
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0_ShmooResults
2_strgval_AAAA
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0^LEGEND^A
2_strgval_failures:failures_burstoff_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0,xxHPCC_DPIN_Dig_slcA_AA1
```

Above is an example of ituff outputs for a 4x4 2D shmoo with all passes, a 1D shmoo with all passes, and a 1D shmoo with all failures.

As of Prime 9.1, ShmooHub prints in the the format:

```
2_tname_{tname}_SSTP
2_strgval_X_{XAxisInitialValue}_Y_{YAxisInitialValue}
2_tname_{tname}_ShmooParams
2_strgval_drive_{XAxisParamName}^{XAxisStart}^{XAxisStop}^{XAxisResolution}^{YAxisParamName}^{YAxisStar}
2_tname_{tname}_ShmooResults
2_strgval_{Results}
2_tname_{tname}^LEGEND^{ResultsFailCharacters}
2_strgval_{LegendInfo}
```

Example2 (ExecutionMode = "PerPin"; ShmooPins =
"xxHPCC_DPIN_Dig_slcA_AA0,xxHPCC_DPIN_Dig_slcA_AA1"):

```

2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalPerPinFailuresShmooHubPrint_P2_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalPerPinFailuresShmooHubPrint_P2_ShmooParams
2_strgval_AriesX^0^3E-06^1E-06^AriesY^0^3E-06^1E-06_ShmooHub
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalPerPinFailuresShmooHubPrint_P2_xxHPCC_DPIN_Dig_slcA_AA0_ShmooResults
2_strgval_BBB*_CC*_E***_****
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalPerPinFailuresShmooHubPrint_P2_xxHPCC_DPIN_Dig_slcA_AA1_ShmooResults
2_strgval_AAA*_DD*_****_****

```

Above is an example of ituff outputs for a 4x4 2D shmoo with some points pass and some points fail.

As of Prime 11.1, ShmooHub prints in the the format:

```

2_tname_{tname}_SSTP
2_strgval_X_{XAxisInitialValue}_Y_{YAxisInitialValue}
2_tname_{tname}_ShmooParams
2_strgval_drive_{XAxisParamName}^{XAxisStart}^{XAxisStop}^{XAxisResolution}^{YAxisParamName}^{YAxisStar}
2_tname_{tname}_{shmoo_pin}_ShmooResults
2_strgval_{Results}
2_tname_{tname}^LEGEND^ResultsFailCharacters}
2_strgval_{LegendInfo}

```

ECADS Example:

Example1 (ExecutionMode = "AllPin"):

```

2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalFailuresEcadsPrint_F0
2_comnt_Plot3Start_Shmoo::PrimeShmooTestMethodTwoDimensionalFailuresEcadsPrint_F0
2_comnt_PLOT_PXName,EcadsX
2_comnt_PLOT_PYName,EcadsY
2_comnt_PLOT_PXStart,0
2_comnt_PLOT_PYStart,0
2_comnt_PLOT_PXStop,3E-06
2_comnt_PLOT_PYStop,3E-06
2_comnt_PLOT_PXStep,3
2_comnt_PLOT_PYStep,3
2_comnt_PLOT_PXValue,1E-08
2_comnt_PLOT_PYValue,5E-09
2_comnt_P3Data_AA**
2_comnt_P3Data_B***
2_comnt_P3Data_****
2_comnt_P3Data_****
2_comnt_P3Legend_A_failures:failures_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_P3Legend_B_failures:failures_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_Plot3End_Shmoo::PrimeShmooTestMethodTwoDimensionalFailuresEcadsPrint_F0

```

Above is an example of an ituff output for a 4x4 2D shmoo.

Tags Include:

2_comnt_PLOT_PXName,<Name of X parameter>

2_comnt_PLOT_PYName,<Name of Y parameter>

2_comnt_PLOT_PXStart,<Value where X starts>

2_comnt_PLOT_PYStart,<Value where Y starts>

2_comnt_PLOT_PXStop,<Value where X ends>

2_comnt_PLOT_PYStop,<Value where Y ends>

2_comnt_PLOT_PXStep,<number of steps X Takes not including first point>

2_comnt_PLOT_PYStep,<number of steps Y Takes not including first point>

2_comnt_PLOT_PXValue,<Initial value of X before Shmoo was ran>

2_comnt_PLOT_PYValue,<Initial value of Y before Shmoo was ran>

Example2 (ExecutionMode = "PerPin"; ShmooPins =

"xxHPCC_DPIN_Dig_slcA_AA0,xxHPCC_DPIN_Dig_slcA_AA1"):

```
2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2
2_comnt_Plot3Start_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2_perpin_xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_PLOT_PXName,AriesX
2_comnt_PLOT_PYName,AriesY
2_comnt_PLOT_PXStart,0
2_comnt_PLOT_PYStart,0
2_comnt_PLOT_PXStop,3E-06
2_comnt_PLOT_PYStop,3E-06
2_comnt_PLOT_PXStep,3
2_comnt_PLOT_PYStep,3
2_comnt_PLOT_PXValue,1E-08
2_comnt_PLOT_PYValue,5E-09
2_comnt_P3Data_BBB*
2_comnt_P3Data_CC**
2_comnt_P3Data_E***
2_comnt_P3Data_****
2_comnt_P3Legend_A_failures:failures_plist:DomainA_All_DPIN_Dig(1001,101,-1,-1):xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_P3Legend_B_failures:failures_plist:DomainA_All_DPIN_Dig(1101,102,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_P3Legend_C_failures:failures_plist:DomainA_All_DPIN_Dig(101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_P3Legend_D_failures:failures_plist:DomainA_All_DPIN_Dig(1101,90,-1,-1):xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_P3Legend_E_failures:failures_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_Plot3End_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2

2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2
2_comnt_Plot3Start_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2_perpin_xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_PLOT_PXName,AriesX
2_comnt_PLOT_PYName,AriesY
2_comnt_PLOT_PXStart,0
2_comnt_PLOT_PYStart,0
2_comnt_PLOT_PXStop,3E-06
2_comnt_PLOT_PYStop,3E-06
2_comnt_PLOT_PXStep,3
2_comnt_PLOT_PYStep,3
2_comnt_PLOT_PXValue,1E-08
2_comnt_PLOT_PYValue,5E-09
2_comnt_P3Data_AAA*
2_comnt_P3Data_DD**
2_comnt_P3Data_****
2_comnt_P3Data_****
2_comnt_P3Legend_A_failures:failures_plist:DomainA_All_DPIN_Dig(1001,101,-1,-1):xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_P3Legend_B_failures:failures_plist:DomainA_All_DPIN_Dig(1101,102,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_P3Legend_C_failures:failures_plist:DomainA_All_DPIN_Dig(101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_P3Legend_D_failures:failures_plist:DomainA_All_DPIN_Dig(1101,90,-1,-1):xxHPCC_DPIN_Dig_slcA_AA1
2_comnt_P3Legend_E_failures:failures_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA0
2_comnt_Plot3End_Shmoo::PrimeShmooTestMethod2DPerPinFailuresEcads_P2
```

Above is an example of an ituff output for a 4x4 2D shmoo.

Tags Include:

2_comnt_PLOT_PXName,<Name of X parameter>

2_comnt_PLOT_PYName,<Name of Y parameter>

2_comnt_PLOT_PXStart,<Value where X starts>

2_comnt_PLOT_PYStart,<Value where Y starts>

2_comnt_PLOT_PXStop,<Value where X ends>

2_comnt_PLOT_PYStop,<Value where Y ends>

2_comnt_PLOT_PXStep,<number of steps X Takes not including first point>

2_comnt_PLOT_PYStep,<number of steps Y Takes not including first point>

2_comnt_PLOT_PXValue,<Initial value of X before Shmoo was ran>

2_comnt_PLOT_PYValue,<Initial value of Y before Shmoo was ran>

ARIES Example:

Example1 (ExecutionMode = "AllPin"):

```
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalFailuresAriesPrint_F0_SSTP
2_strgval_X_10e-9S
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalFailuresAriesPrint_F0_drive_var1_0_3E-06_1E-06
2_rawbinary_msbF_0001
```

Above is an example of an ituff output for a 1x4 1D shmoo.

```
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalFailuresAriesPrint_F0_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalFailuresAriesPrint_F0_AriesX_0_3E-06_1E-06_AriesY_0_3E-06_1E-06
2_rawbinary_msbF_0011
2_rawbinary_msbF_0111
2_rawbinary_msbF_1111
2_rawbinary_msbF_1111
```

Above is an example of an ituff output for a 4x4 2D shmoo.

Example2 (ExecutionMode = "PerPin"; ShmooPins =
"xxHPCC_DPIN_Dig_slcA_AA0,xxHPCC_DPIN_Dig_slcA_AA1"):

```
2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresAries_P2_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresAries_P2_xxHPCC_DPIN_Dig_slcA_AA0_AriesX_0_3E-06_1E-06_AriesY_0_3E-06_1E-06
2_rawbinary_msbF_0001
2_rawbinary_msbF_0011
2_rawbinary_msbF_0111
2_rawbinary_msbF_1111

2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresAries_P2_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethod2DPerPinFailuresAries_P2_xxHPCC_DPIN_Dig_slcA_AA1_AriesX_0_3E-06_1E-06_AriesY_0_3E-06_1E-06
2_rawbinary_msbF_0001
2_rawbinary_msbF_0011
2_rawbinary_msbF_1111
2_rawbinary_msbF_1111
```

Above is an example of an ituff output for a 4x4 2D shmoo.

Custom User Code Hooks

Shmoo test method supports the following extensions:

Extensions used in verify:

```

/// <summary>
/// Class to define extendable methods.
/// </summary>
public interface IShmooExtensions
{
    /// <summary>
    /// Method to be implemented by user to return all the points in the x-axis range that TestMeth
    /// </summary>
    /// <param name="pointValue">string from test method parameter "XAxisRange". It usually contain
    /// so user can generate the list of double points for this axis. User is responsible for string
    /// <returns>List of double points values for x-axis to iterate over.</returns>
    List<double> GetXAxisPoints(string pointValue);

    /// <summary>
    /// Method to be implemented by user to return all the points in the y-axis range that TestMeth
    /// </summary>
    /// <param name="pointValue">string from test method parameter "YAxisRange". It usually contain
    /// so user can generate the list of double points for this axis. User is responsible for string
    /// <returns>List of double points values for y-axis to iterate over.</returns>
    List<double> GetYAxisPoints(string pointValue);

    /// <summary>
    /// Extension to allow modifying of plist execution settings, for example set capture pins or d
    /// </summary>
    /// <param name="patlist">The plist name to build the IFunctionalTest.</param>
    /// <param name="levelsTc">The levels test condition name to build the IFunctionalTest.</param>
    /// <param name="timingsTc">The timings test conditions name to build the IFunctionalTest.</param>
    /// <param name="prePlist">The callback to run before the plist execution.</param>
    /// <returns>The IFunctionalTest object build as per user requirements.</returns>
    IFunctionalTest GetFunctionalTest(string patlist, string levelsTc, string timingsTc, string prePlist);
}

```

Extensions used in Execute



```

/// <summary>
/// Extension to allow user to run some code prior to plist execution on specific shmoo point.
/// </summary>
/// <param name="point">Shmoo point to apply.</param>
/// <param name="functionalTest">Func test object that contains execution data.</param>
/// <returns>true if apply succeeded and this point is relevant for the test, and false if this
bool PrePointExecute(ShmooPoint point, IFunctionalTest functionalTest);

/// <summary>
/// Extension to allow user to run some code after plist execution on specific shmoo point. The
/// </summary>
/// <param name="point">Current shmoo point.</param>
/// <param name="functionalTest">Func test object that contains execution data.</param>
void PostPointExecute(ShmooPoint point, IFunctionalTest functionalTest);

/// <summary>
/// Extension allow user to override the code that runs on a specific shmoo point. By default,
/// The return value of this function will impact the exit port of the instance.
/// </summary>
/// <param name="point">Current shmoo point.</param>
/// <param name="results">The object for holding the exit port, and the normal/regional function
void PointExecute(ShmooPoint point, ShmooTestInstanceResults results);

/// <summary>
/// Extension to allow user to run some code after all shmoo points have executed, and process
/// By default, if at least one shmoo point fails, then the instance will exit from port 0. Ot
/// </summary>
/// <param name="functionalTest">Func test object that contains execution data.</param>
/// <param name="results">The object for holding the exit port, and the normal/regional function
void PostExecute(IFunctionalTest functionalTest, ShmooTestInstanceResults results);

/// <summary>
/// Extension to allow user to run some code before any shmoo points have executed.
/// </summary>
/// <param name="functionalTest">Func test object that contains execution data.</param>
void PreExecute(IFunctionalTest functionalTest);

/// <summary>
/// Called prior to any test in Execute to generate the test instance results object.
/// </summary>
/// <returns>The test instance results object.</returns>
ShmooTestInstanceResults CreateTestInstanceResults();

/// <summary>
/// Gets the list of pins to mask execution after execution. The test method will merge this li
/// </summary>
/// <returns>The list of mask pins.</returns>
List<string> GetDynamicPinMask();
}

```

Optional Features

User Defined Shmoo Axis

A user defined axis is activated when the user provides an input of UserDefined for the X/YAxisParamType input for their chosen axis.

By default, a UserDefined axis will do nothing during any execution phase besides print to debug that nothing is being done during each phase. For users to overwrite default functionality, the extensions mentioned previously should be used.

Below is an example snippet of usercode which is overwriting the default shmoo behavior by adding new user defined types. Note that the new type names are loaded in via user defined parameters "XUserDefinedType" and "YAxisUserDefinedType"

Main Usercode file:



```

[PrimeTestMethod]
public class SHM00_UC : PrimeShmooTestMethod, IShmooExtensions
{
    private ICustomShmooAxis xAxis;
    private ICustomShmooAxis yAxis;

    /// <summary>
    /// Enum for X/Y Axis Type parameters. Used for special user defined types.
    /// </summary>
    public enum UserDefinedAxisType
    {
        /// <summary>
        /// Axis is unused.
        /// </summary>
        None,

        /// <summary>
        /// First custom type.
        /// </summary>
        UserAxis1,

        /// <summary>
        /// Second custom type.
        /// </summary>
        UserAxis2,
    }

    /// <summary>
    /// Gets or sets X parameter type.
    /// </summary>
    public UserDefinedAxisType XAxisUserDefinedType { get; set; } = UserDefinedAxisType.None;

    /// <summary>
    /// Gets or sets Y parameter type.
    /// </summary>
    public UserDefinedAxisType YAxisUserDefinedType { get; set; } = UserDefinedAxisType.None;

    /// <summary>
    /// Initializes custom Shmoo Axis.
    /// </summary>
    public override void CustomVerify()
    {
        this.xAxis = this.GetShmooAxis(this.XAxisUserDefinedType);
        this.yAxis = this.GetShmooAxis(this.YAxisUserDefinedType);
    }

    /// <inheritdoc/>
    bool IShmooExtensions.PointExecute(ShmooPoint point)
    {
        bool result = this.xAxis.PointExecute(point.XValue, "X");
        result &= this.yAxis.PointExecute(point.YValue, "Y");

        return result;
    }

    private ICustomShmooAxis GetShmooAxis(UserDefinedAxisType type)
    {
        switch (type)
        {
            case UserDefinedAxisType.None:
                return new EmptyAxisType();
            case UserDefinedAxisType.UserAxis1:
                return new UserAxisType1();
            case UserDefinedAxisType.UserAxis2:
                return new UserAxisType2();
        }

        return new EmptyAxisType();
    }
}

```

```

    }
}

```

Custom AxisClass definitions:



```

/// <summary>
/// Initializes a custom shmoo axis.
/// </summary>
internal interface ICustomShmooAxis
{
    /// <summary>
    /// To be executed on each shmoo point.
    /// </summary>
    /// <param name="point">point value to be executed.</param>
    /// <param name="axisName">Axis name for print.</param>
    /// <returns>true if successful execution, false otherwise.</returns>
    bool PointExecute(double point, string axisName);
}

/// <summary>
/// First custom axis type.
/// </summary>
internal class UserAxisType1 : ICustomShmooAxis
{
    /// <inheritdoc/>
    public bool PointExecute(double point, string axisName)
    {
        // Custom point execute code goes here. Example is provided
        var toPrint = Prime.Services.DatalogService.GetItuffComntWriter();
        toPrint.AddData($"UserAxisType1PointExecute_{axisName}_{point}");
        Prime.Services.DatalogService.WriteToItuff(toPrint);
        return true;
    }
}

/// <summary>
/// Second custom axis type.
/// </summary>
internal class UserAxisType2 : ICustomShmooAxis
{
    /// <inheritdoc/>
    public bool PointExecute(double point, string axisName)
    {
        // Custom point execute code goes here. Example is provided
        var toPrint = Prime.Services.DatalogService.GetItuffComntWriter();
        toPrint.AddData($"UserAxisType2PointExecute_{axisName}_{point}");
        Prime.Services.DatalogService.WriteToItuff(toPrint);
        return true;
    }
}

```

Mtpl:



```
Test SHMOO_UC PrimeShmooTestMethod2DUserCode_P1
{
    LevelsTc = "basic_func_lvl_nom";
    Patlist = "passing_plist";
    TimingsTc = "basic_func_timing_10MHz_20MHz";
    XAxisParam = "xparam";
    XAxisRange = "1:5:1";
    YAxisParam = "yparam";
    YAxisRange = "1:5:1";
    XAxisParamType = "UserDefined";
    YAxisParamType = "UserDefined";
    PowerDownBetweenPoints = "DISABLED";
    IfeObject = "";
    LogLevel = "Enabled";

    XAxisUserDefinedType = "UserAxis1";
    YAxisUserDefinedType = "UserAxis2";
}
```

```
Test SHMOO_UC PrimeShmooTestMethod1DUserCode_P1
{
    LevelsTc = "basic_func_lvl_nom";
    Patlist = "passing_plist";
    TimingsTc = "basic_func_timing_10MHz_20MHz";
    XAxisParam = "xparam";
    XAxisRange = "1:5:1";
    XAxisParamType = "UserDefined";
    PowerDownBetweenPoints = "DISABLED";
    IfeObject = "";
    LogLevel = "Enabled";

    XAxisUserDefinedType = "UserAxis1";
}
```

The code and mtpl above will allow users to print a comment dependent on the type of user axis to ituff. an actual code example is provided in the SampleTp included in UserSDK

Voltage Configuration Axis

A voltage axis has a special format for input parameters which is made up of subparameters denoted by --{subparametername}=

Each subparameter is described as follows:

Sub Parameter Name	Required?	Type	Comments
domains	Yes	space separated list	Domains to be used to create internal voltage object. Domains should be defined in separate Aleph configuration file.
fivrcondition	Yes	string	FIVR condition name. Defined in ALEPH configuration file."
overrides	No	string	Voltage overrides. Overrides should be in the format: "domain:value,...,domainN:valueN"
dlvrpins	No	Space separated list	list of dlvrpins to be used on creation of internal voltage object. Pin names here are checked against shared storage and uservars to populate pin attributes needed for voltage object creation

Example MTPL:

```

Test PrimeShmooTestMethod PrimeShmooTestMethodTwoDimensionVoltageConfigFivrAndDlvr_P1
{
    Patlist = "Fivr_Plist";
    TimingsTc = "Shmoo::basic_func_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::basic_func_lvl_nom";
    XAxisParam = "--domains=FIVR_VLOAD_CORE0_LC --fivrcondition=FIVR_VLOAD_CORE01_Literal";
    XAxisRange = "0:1:4";
    XAxisParamType = "VoltageConfig";
    YAxisParam = "--domains=FIVR_VLOAD_CORE0_LC --fivrcondition=FIVR_VLOAD_CORE01_Literal --dlvrpins=HDD";
    YAxisRange = "0:1:4";
    YAxisParamType = "VoltageConfig";
    IfeObject = "";
    LogLevel = "Enabled";
}

Test PrimeShmooTestMethod PrimeShmooTestMethodOneDimensionVoltageConfigWithOverrides_P1
{
    Patlist = "Fivr_Plist";
    TimingsTc = "Shmoo::basic_func_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::basic_func_lvl_nom";
    XAxisParam = "--domains=FIVR_VLOAD_CORE0_LC --fivrcondition=FIVR_VLOAD_CORE01_Literal --overrides=FI";
    XAxisRange = "0:1:4";
    XAxisParamType = "VoltageConfig";
    IfeObject = "";
    LogLevel = "Enabled";
}

```

Regional Kill

Regional kill mode is activated when the user provides an input for RegionalKillLimits in the format: "XMin, XMax, YMin, YMax".

The Minimum and maximum points are used as a boundary that points are compared to inclusively. If there is a failure within the boundary, the test will exit port 2, otherwise it will exit port 3.

```
2_comnt_PLOT_PXName,drive_var1
2_comnt_PLOT_PYName,drive_var2
2_comnt_PLOT_PXStart,8E-09
2_comnt_PLOT_PYStart,8E-09
2_comnt_PLOT_PXStop,1.2E-08
2_comnt_PLOT_PYStop,1.2E-08
2_comnt_PLOT_PXStep,1E-09
2_comnt_PLOT_PYStep,1E-09
2_comnt_PLOT_PXValue,2E-09
2_comnt_PLOT_PYValue,1E-09
2_comnt_P3Data_AAAAA
2_comnt_P3Data_A***A
2_comnt_P3Data_A***A
2_comnt_P3Data_A***A
2_comnt_P3Data_AAAAA
2_comnt_P3Legend_A_failures:startPatternValidation_plist:HBI_DPIN_ALL_domain(1,1,-1):xxHBI_DPIN_slc2_A097,xxHBI_DPIN_slc2_A101,xxHBI_DPIN_slc3_A117,xxHBI_DPIN_slc3_A139
2_comnt_Plot3End_Shmoo::RegionalKillFailOutsideRange_P3
```

Above is an example of an ituff output for a 3x3 regional kill region for a 2D shmoo with failures outside of the limits. This test would exit port 3;

```
2_comnt_PLOT_PXName,drive_var1
2_comnt_PLOT_PYName,drive_var2
2_comnt_PLOT_PXStart,8E-09
2_comnt_PLOT_PYStart,8E-09
2_comnt_PLOT_PXStop,1.2E-08
2_comnt_PLOT_PYStop,1.2E-08
2_comnt_PLOT_PXStep,1E-09
2_comnt_PLOT_PYStep,1E-09
2_comnt_PLOT_PXValue,2E-09
2_comnt_PLOT_PYValue,1E-09
2_comnt_P3Data_AAAAA
2_comnt_P3Data_A***A
2_comnt_P3Data_A***A
2_comnt_P3Data_A***A
2_comnt_P3Data_AAAAA
2_comnt_P3Legend_A_failures:startPatternValidation_plist:HBI_DPIN_ALL_domain(1,1,-1):xxHBI_DPIN_slc2_A097,xxHBI_DPIN_slc2_A101,xxHBI_DPIN_slc3_A117,xxHBI_DPIN_slc3_A139
2_comnt_Plot3End_Shmoo::RegionalKillFailRightInside_P2
```

Above is an example of an ituff output for a 3x3 regional kill region for a 2D shmoo with failures inside of the limits. This test would exit port 2.

Tname Scale customization

By default, values will be printed using scientific notation in their base units (seconds, volts, etc).

To print values to ituff using non-base units (nanoseconds, millicolts, etc), users should provide an their desired scalar for the respective axis using the X/YAxisDatalogPrefix parameters.

This mode is useful to prevent load issues with XEUS database, as XEUS does not allow the '-' character within Tnames

Example of mtpl using non-default scalar on output:

mtpl file:

```

Test PrimeShmooTestMethod PrimeShmooTestMethodOneDimensionalWithFailures_F0
{
    Patlist = "failures_burstoff_plist";
    TimingsTc = "Shmoo::basic_func_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::basic_func_lvl_nom";
    XAxisParam = "drive_var1";
    XAxisRange = "0:0.000001:4";
    XAxisParamType = "TimingTestCondition";
    XAxisDatalogPrefix = "Micro";
    YAxisParam = "";
    YAxisRange = "";
    PowerDownBetweenPoints = "DISABLED";
    IfeObject = "";
    LogLevel = "Enabled";
}

```

Datalog Output:

```

2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0_SSTP
2_strgval_X_10e-9S
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0^drive_var1^0^3^1_ShmooHub
2_strgval_AAAA
2_tname_Shmoo::PrimeShmooTestMethodOneDimensionalWithFailures_F0^LEGEND^A
2_strgval_failures:failures_burstoff_plist:DomainA_All_DPIN_Dig(1101,100,-1,-1):xxHPCC_DPIN_Dig_slcA_AA

```

As can be seen, the scalar is applied to the Tname value for drive_var1, converting it from a scale of 0.000001 to 1

Axis name customization

By default, axes will use their parameter name as the axis name during tname prints. This can cause with strgval format's 128 character limit on tnames if parameter names are exceedingly long or when they include a list of parameters.

Users can customize an axis' name for datalog by using the X/YAxisDatalogName parameters.

Example of mtpl using custom names for X and Y axis:

mtpl file:

```

Test PrimeShmooTestMethod PrimeShmooTestMethodTwoDimensionalCustomAxisNames_P1
{
    Patlist = "startPatternValidation_plist";
    TimingsTc = "Shmoo::basic_functional_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::SampleFunctionalTestMethodTC";
    XAxisParam = "drive_var1";
    XAxisRange = "8e-9:1e-9:4";
    YAxisParam = "drive_var2";
    XAxisParamType = "TimingTestCondition";
    YAxisRange = "8e-9:1e-9:4";
    YAxisParamType = "TimingTestCondition";
    XAxisDatalogName = "CustomXName";
    YAxisDatalogName = "CustomYName";
    PowerDownBetweenPoints = "DISABLED";
    IfeObject = "";
    LogLevel = "Enabled";
}

```

Datalog Output:

```
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalCustomAxisNames_P1_SSTP
2_strgval_X_10e-9S_Y_5e-9S
2_tname_Shmoo::PrimeShmooTestMethodTwoDimensionalCustomAxisNames_P1^CustomXName^0^3E-06^1E-06^CustomYName
2_strgval_****_****_****_****
```

This mode is useful for shortening tnames to below the 128 character limit for certain test plans.

TPL Samples

Here are a few test instance examples using the Shmoo test method:

```
Test PrimeShmooTestMethod OneDimensionalShmoo_P1
{
    Patlist = "startPatternValidation_plist";
    TimingsTc = "Shmoo::basic_functional_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::SampleFunctionalTestMethodTC";
    XAxisParam = "drive_var1";
    XAxisRange = "8e-9:1e-9:4";
    XAxisDatalogPrefix = "Nano";
    XAxisParamType = "TimingTestCondition";
    YAxisParam = "";
    YAxisRange = "";
    PowerDownBetweenPoints = "ENABLED";
    IfeObject = "";
    LogLevel = "Enabled";
}

Test PrimeShmooTestMethod TwoDimensionalShmoo_P1
{
    Patlist = "startPatternValidation_plist";
    TimingsTc = "Shmoo::basic_functional_timing_10MHz_20MHz";
    LevelsTc = "Shmoo::SampleFunctionalTestMethodTC";
    XAxisParam = "drive_var1";
    XAxisRange = "8e-9:1e-9:4";
    XAxisDatalogPrefix = "Nano";
    YAxisParam = "drive_var2";
    XAxisParamType = "TimingTestCondition";
    YAxisRange = "8e-9:1e-9:4";
    YAxisDatalogPrefix = "Nano";
    YAxisParamType = "TimingTestCondition";
    PowerDownBetweenPoints = "DISABLED";
    IfeObject = "";
    LogLevel = "Enabled";
}
```

Exit Ports

The Shmoo test method supports the following exit ports:

Exit Port	Condition	Description
-2	Alarm	Any alarm condition
-1	Error	Any software condition error
0	Fail	Execution has failed for all Shmoo points.
1	Pass	Execution has passed for all Shmoo points.
2	Pass	Execution has passed for at least one point and has failed for at least one point.
3	Pass	Regional kill mode was used and failure was not found inside the limits.

Additional Dependencies

More dependencies to consider for this TestMethod to well operate:

- N/A.

Version tracking

Date	Version	Author	Comments
Jan 3 rd , 2021	1.0.0	Wajde Dakwar	Initial Version.
Jun 9 th , 2021	1.0.1	Kevin D. Krake	Adding pin masking ability.
Dec 19 th , 2021	1.0.2	Lauren McDonald	Refactoring to require explicit axis types in order make future type additions easier
Jan 14 th , 2022	1.0.2	Lauren McDonald	Changing names of parameters to prevent overlap with existing usercode
Feb 9 th , 2022	1.0.3	Lauren McDonald	Adding ECADS print mode
Feb 11 th , 2022	1.0.4	Lauren McDonald	Adding patconfig and patconfig setpoint support
Feb 24 th , 2022	1.0.5	Lauren McDonald	Adding Regional kill
Apr 8 th , 2022	1.0.6	Lauren McDonald	Adding Voltage type axis
Apr 9 th , 2022	1.1.0	Kevin D Krake	Adding new Results method for exit port handling
Apr 28 th , 2022	1.1.1	Lauren McDonald	Changing ShmooHubFormat to match with newly supported Trace Format
May 4 th , 2022	1.1.2	Lauren McDonald	Changing port behavior to match with previous Evergreen behavior
Oct 20 th , 2022	1.1.3	ong, Ping Ping	Adding PerPin Mode feature.

Acronyms

Definition of acronyms used in this document:

- **REP:** Prime Test-Method Specification
- **TPL:** Test Programming Language